

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №2

Тема: Основи проектування

Виконав:
студент групи ІА-34
Єресько І.С
Перевірів:
Мягкий М.Ю

Київ-2025

Тема: Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server) Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна).

Мета: Обрати зручну систему побудови UML-діаграм та навчитися будувати діаграми варіантів використання для системи що проєктується, розробляти сценарії варіантів використання та будувати діаграми класів предметної області.

1.Короткі теоретичні відомості

У даній роботі було виконано проєктування системи PowerShell Terminal з використанням UML-діаграм та моделювання її функціональності й структури. Спочатку були створені діаграми варіантів використання (Use Case Diagram), які застосовано для візуалізації взаємодії користувача з системою. Визначено основного актора - Користувача - та його основні прецеденти: виконання команд і скриптів, управління інтерфейсом та вкладками, збереження результатів і робота з історією команд. Для кожного прецеденту були описані сценарії виконання, передумови, постумови та можливі винятки.

Далі була побудована діаграма класів (Class Diagram), що відображає структуру системи та взаємозв'язки між ключовими елементами. До основних модулів належать MainForm як головний компонент додатку, менеджери TabManager, ThemeManager та HistoryService, а також рівень репозиторіїв: CommandHistoryRepository, UserSettingsRepository та TerminalSessionRepository. До доменної області включено такі моделі, як TerminalTab, TerminalControl, CommandInterpreter і TerminalTheme. Окремо визначено сутності бази даних: CommandHistory, UserSettings, TerminalSession та TerminalDbContext.

Прецеденти системи детально описують основні сценарії роботи: виконання команди PowerShell, створення нової вкладки та зміну теми інтерфейсу.

Для реалізації системи бази даних було вирішено використати SQL Server Management Studio (SSMS). Створена база PowerShellTerminalDB забезпечує зберігання інформації про сесії, історію виконаних команд та налаштування користувачів. Структура бази включає таблицю UserSettings, що містить параметри відображення інтерфейсу (шрифти, кольори, тема, розміри вікна), таблицю TerminalSessions з даними про сесії терміналу та таблицю CommandHistories, де зберігається історія команд разом із результатами їх виконання.

2. Use Case Diagram

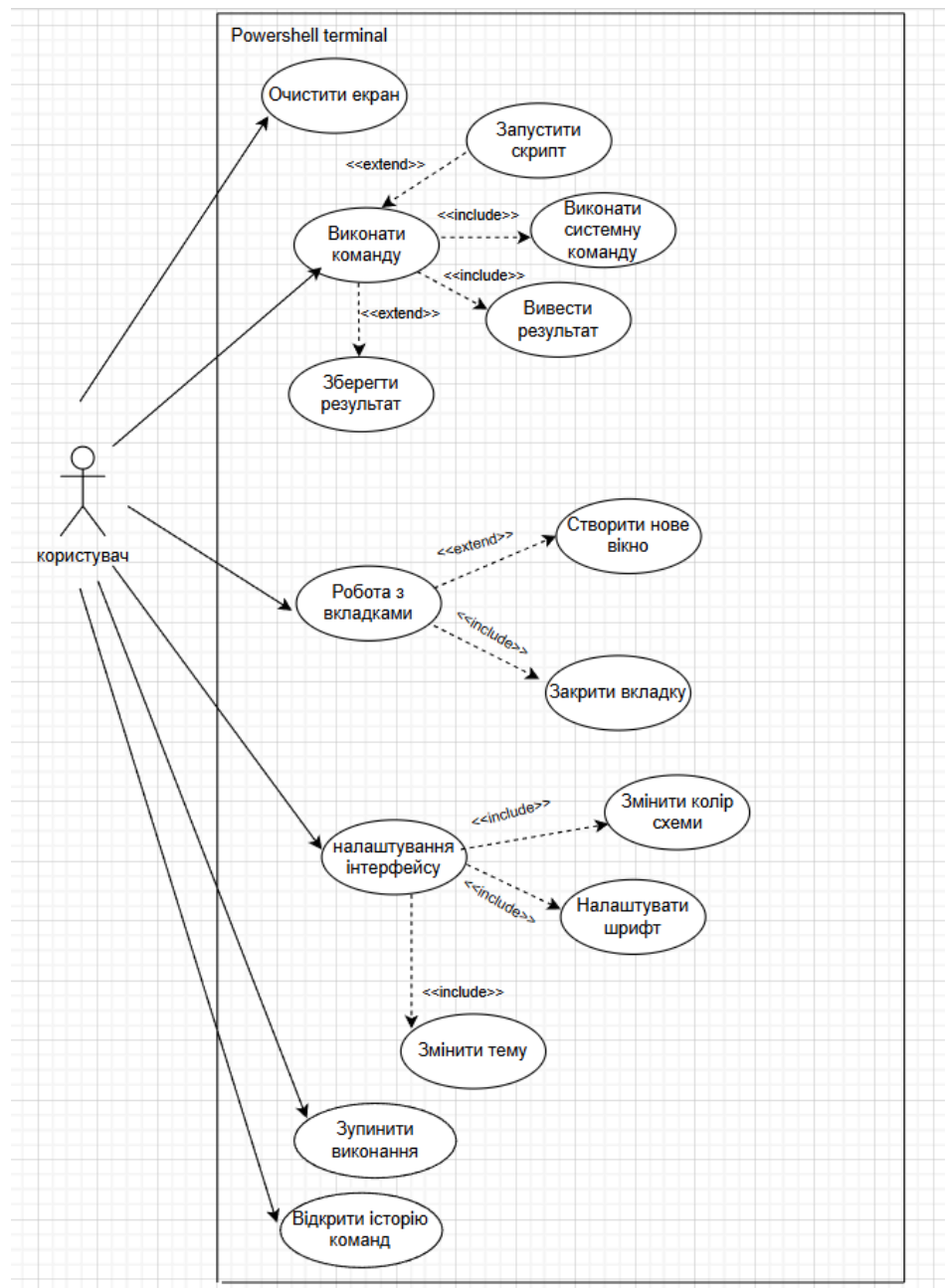


Рис 2.1: Use Case Diagram

Опис Use Case діаграми

Актором системи є Користувач, який взаємодіє з PowerShell Terminal для виконання команд, налаштування інтерфейсу та керування сесіями. Одним із базових варіантів використання є очищення екрана: користувач очищає вікно терміналу від попередніх результатів, після чого екран стає порожнім і готовим до введення нових команд. Інший важливий сценарій - запуск скрипта, який розширює варіант «Виконати команду» і описує запуск заздалегідь підготовленого PowerShell-скрипта.

Основним прецедентом виступає виконання команди, що реалізує базову функціональність терміналу - обробку введених користувачем інструкцій. Цей варіант включає дії «Виконати системну команду» та «Вивести результат» і розширює варіант «Зберегти результат». Виконання системної команди описує роботу команд, що взаємодіють із операційною системою, наприклад з файлами або процесами. Після виконання будь-якої команди система виводить результат у термінал, а при потребі користувач може зберегти отримані дані у файл.

Система також підтримує керування інтерфейсом і вкладками. Варіант «Робота з вкладками» дає можливість працювати в кількох робочих областях одночасно; він включає операцію закриття активної вкладки та розширює сценарій «Створити нове вікно». Закриття вкладки відбувається без завершення всієї сесії, а створення нового вікна дозволяє відкрити окрему незалежну термінальну сесію.

Налаштування інтерфейсу представляє окремий варіант використання, у якому користувач може змінювати зовнішній вигляд терміналу. Він включає зміну кольорової схеми, налаштування шрифту та вибір теми оформлення. Зміна кольорової схеми дозволяє підібрати нову палітру інтерфейсу, налаштування шрифту відповідає за його розмір та тип, а зміна теми забезпечує можливість вибору між світлою та темною стилізацією.

Серед додаткових можливостей системи передбачено варіант «Зупинити виконання», який дозволяє перервати поточний процес або скрипт. Також користувач може скористатися функцією відкриття історії команд - для перегляду та повторного виконання раніше введених команд.

3 Діаграма класів

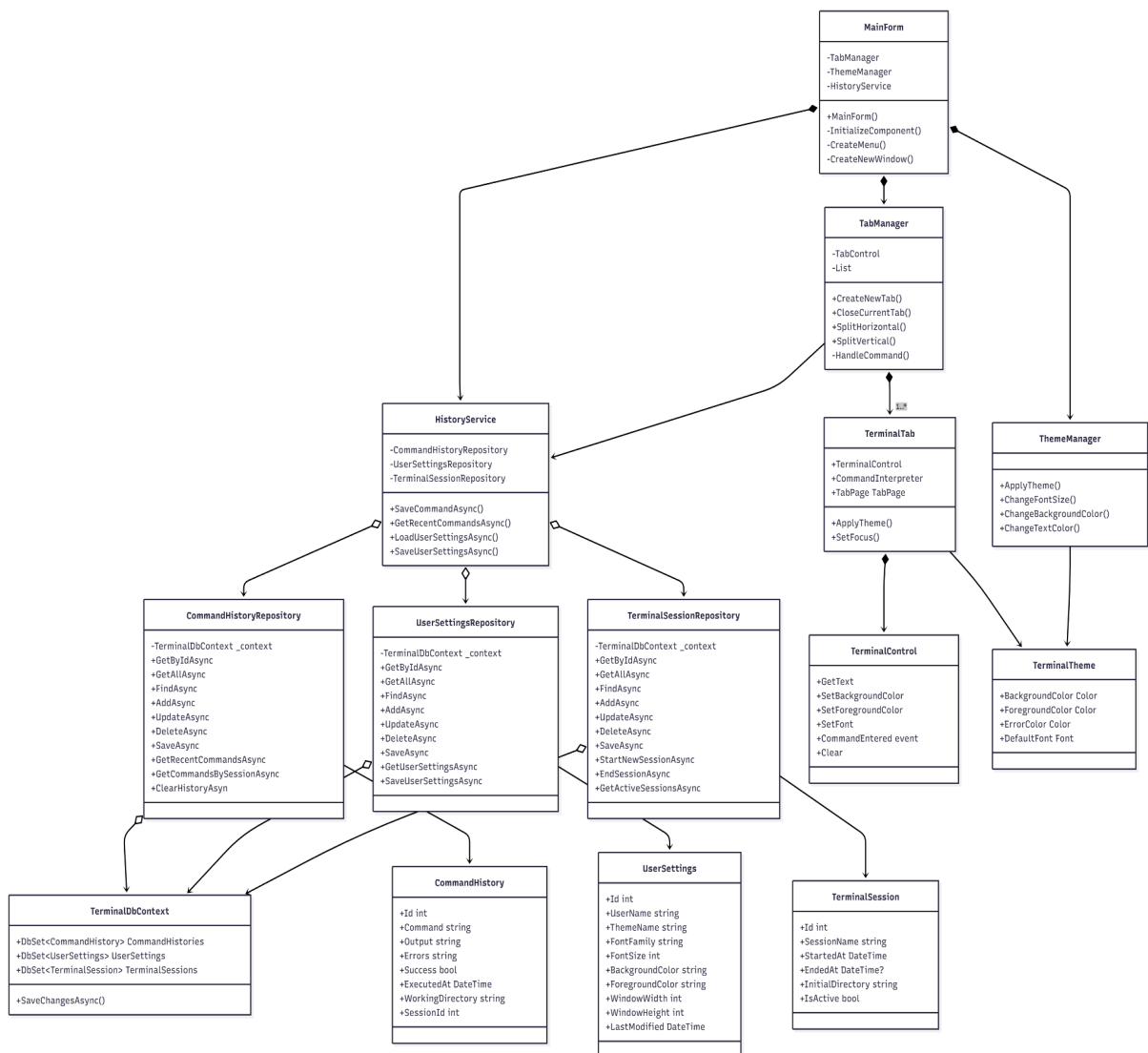


Рис 3.1: Діаграма класів (Class Diagram)

Головна форма

- MainForm - головне вікно програми, координує всі компоненти

Менеджер

- TabManager - керує вкладками (створення, закриття, розділення)

- ThemeManager - керує темами, кольорами, шрифтами

Сервіси та репозиторії

- HistoryService- зберігає/завантажує історію команд та налаштування
- CommandHistoryRepository - робота з історією команд в БД
- UserSettingsRepository - робота з налаштуваннями користувача
- TerminalSessionRepository - робота з сесіями терміналу

Компоненти терміналу

- TerminalTab - окрема вкладка терміналу
- TerminalControl - власне поле введення/виводу команд
- CommandInterpreter - інтерпретатор команд (виконує команди)
- TerminalTheme - налаштування кольорів та шрифтів

Сутності бази даних

- CommandHistory - запис про виконану команду
- UserSettings - налаштування користувача
- TerminalSession - сесія роботи з терміналом
- TerminalDbContext - підключення до бази даних

Шаблон Repository

Окремі класи для кожної сутності (CommandHistoryRepository, UserSettingsRepository, TerminalSessionRepository)

Інкапсуляція доступу до БД - вся робота з базою через репозиторії

CRUD методи - GetById, GetAll, Add, Update, Delete

Спеціалізовані методи - GetRecentCommandsAsync, GetUserSettingsAsync тощо

Потік роботи

1. MainForm запускає TabManager для створення вкладок
2. Кожна TerminalTab містить TerminalControl для вводу команд
3. CommandInterpreter виконує команди
4. HistoryService зберігає результати через репозиторії в БД
5. ThemeManager застосовує вигляди до всіх компонентів

3.Прецеденти

Прецедент 1: Виконання PowerShell команди

Актор: Користувач

Ціль: Виконати команду PowerShell та отримати результат

Передумови: Термінал запущено

Основний сценарій:

- Користувач вводить команду у поле вводу
- Система передає команду інтерпретатору
- Інтерпретатор вибирає стратегію виконання
- Виконується команда через PowerShell
- Результат виводиться у термінал

Постумови: Команда виконана, результат відображено

Вийнятки:

1а. Команда введена з помилкою - система виводить повідомлення про синтаксичну помилку

1б. Відсутні права доступу - система повідомляє про відмову у доступі

1в. PowerShell команди не існує - система показує повідомлення про неможливість виконання

Прецедент 2: Створення нової вкладки

Актор: Користувач

Ціль: Створити нову сесію терміналу у вкладці

Передумови: Головне вікно відкрите

-Основний сценарій:

- Користувач обирає "File - New Tab"
- Система створює нову вкладку
- Застосовується поточна тема
- Нова вкладка стає активною

Постумови: Створено нову вкладку терміналу

Вийнятки:

- 2а. Недостатньо пам'яті для створення нової вкладки - система виводить повідомлення про помилку
- 2б. Помилка ініціалізації контролю терміналу - вкладка не створюється, система повідомляє користувача
- 2в. Система повертає null - система скасовує створення вкладки

Прецедент 3: Зміна теми інтерфейсу

Актор: Користувач

Ціль: Змінити колірну схему терміналу

Передумови: Термінал запущено

Основний сценарій:

- Користувач обирає "View - Dark Theme"
- Система створює нову тему
- Застосовує тему до всіх відкритих вкладок
- Оновлює кольори тексту та фону

Постумови: Тема змінена для всіх вкладок

Вийнятки:

- 3а. Обрана тема відсутня або пошкоджена - система повертає стандартну тему
- 3б. Помилка оновлення стилів - система частково оновлює інтерфейс і повідомляє користувача
- 3в. Відсутній доступ до файлів теми - система показує повідомлення про неможливість зміни теми

Постумови: Тема змінена для всіх вкладок

4. Структура системи та основні класи

// Main Application

[public class MainForm : Form](#)

// Managers

[public class TabManager](#)

[public class ThemeManager](#)

[public class HistoryService](#)

// Repository Layer

[public class CommandHistoryRepository](#)

[public class UserSettingsRepository](#)

[public class TerminalSessionRepository](#)

// Core Models

[public class TerminalTab](#)

[public class TerminalControl](#)

[public class CommandInterpreter { }](#)

[public class TerminalTheme](#)

// Database Entities

[public class CommandHistory](#)

[public class UserSettings](#)

[public class TerminalSession](#)

[public class TerminalDbContext](#)

5. Система баз даних.

Для реалізації цієї бази даних було вирішено використовувати SQL Server Management Studio (SSMS). Це дозволяє ефективно створювати та адмініструвати таблиці, встановлювати зв'язки між ними, виконувати SQL-запити та керувати даними у структурованому середовищі. Використання SSMS забезпечує зручний графічний інтерфейс для розробки бази даних та управління сесіями і історією команд.

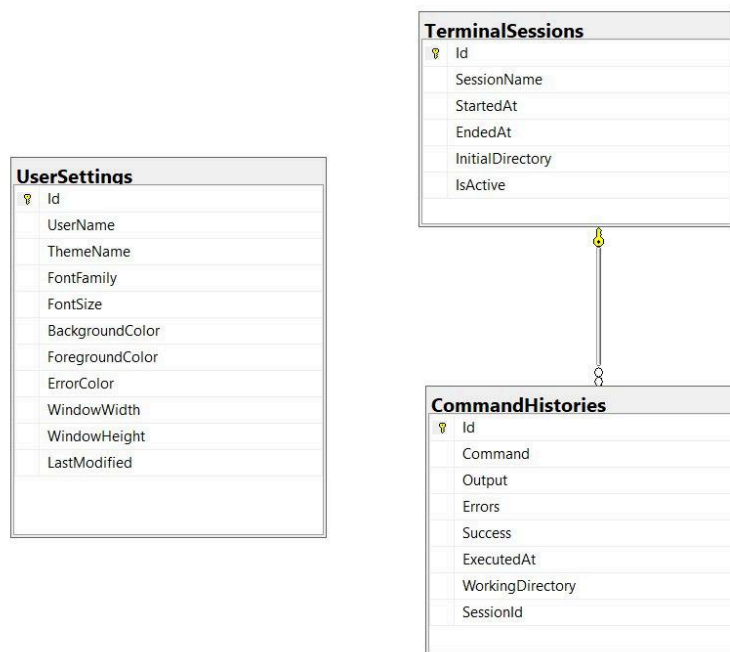


Рис 5.1:База даних (PowerShellTerminalDB)

Опис бази даних

Назва БД: (PowerShellTerminalDB)

Мета: Зберігання даних про сесії терміналу, історію команд та налаштування користувачів.

Таблиці

1. UserSettings

Зберігає налаштування користувачів терміналу.

Поля:

- **Id** - унікальний ідентифікатор користувача.
- **UserName** - ім'я користувача.
- **ThemeName** - назва теми оформлення.
- **FontFamily** - шрифт терміналу.
- **FontSize** - розмір шрифту.
- **BackgroundColor** - колір фону.
- **ForegroundColor** - колір тексту.
- **ErrorColor** - колір для повідомлень про помилки.

- WindowWidth - ширина вікна терміналу.
- WindowHeight - висота вікна терміналу.
- LastModified - дата останньої зміни налаштувань.

2. TerminalSessions

Зберігає інформацію про сесії терміналу.

Поля:

- Id - унікальний ідентифікатор сесії.
- SessionName - назва сесії.
- StartedAt - дата та час початку сесії.
- EndedAt - дата та час завершення сесії.
- InitialDirectory - початковий каталог терміналу.
- IsActive - прапорець активності сесії (true/false).

3. CommandHistories

Зберігає історію виконаних команд у сесіях терміналу.

Поля:

- Id - унікальний ідентифікатор запису.
- Command - виконана команда.
- Output - результат виконання команди.
- Errors - помилки виконання (якщо були).
- Success - прапорець успішності виконання.
- ExecutedAt - дата та час виконання команди.
- WorkingDirectory - робочий каталог під час виконання.
- SessionId - зв'язок із таблицею TerminalSessions.

Взаємозв'язки

TerminalSessions з CommandHistories: один-до-багатьох (1:N).

Кожна сесія може містити багато записів історії команд.

Висновок

У ході виконання роботи було здійснено повне проектування системи PowerShell Terminal, що за функціональністю нагадує типовий термінал PowerShell і підтримує виконання команд, запуск скриптів, роботу з кількома вкладками, керування інтерфейсом та зміною теми оформлення.

Створено діаграму варіантів використання, яка відобразила всі ключові взаємодії користувача з терміналом: виконання команд, збереження результатів, налаштування інтерфейсу, роботу з вкладками, відкриття історії та керування сесіями. Для кожного варіанта використання були детально описані сценарії, що дало можливість сформулювати повний функціональний опис системи.

6. Контрольні запитання

1.Що таке UML

UML, або Unified Modeling Language (Уніфікована мова моделювання), є фундаментальним інструментом в арсеналі сучасного розробника програмного забезпечення. Це не просто мова, а стандартизована графічна мова для візуалізації, специфікації, конструювання та документування артефактів програмних систем. Головна сила UML полягає в його здатності наочно представити складну структуру системи, її динамічну поведінку під час виконання, а також взаємодію між різними компонентами та зовнішніми акторами. Використання UML дозволяє команді розробників, архітекторів та замовників знаходити спільну мову, зменшує ризик непорозумінь і створює міцний фундамент для успішної реалізації проекту.

2.Що таке діаграма класів UML?

Діаграма класів в UML є одним з найважливіших та найпоширеніших типів діаграм, який належить до групи структурних діаграм. Вона служить для статичного представлення структури додатку, відображаючи основні будівельні блоки системи – класи, їх внутрішню структуру у вигляді атрибутів (полів) та операцій (методів), а також найрізноманітніші зв'язки між цими класами. Саме на цій діаграмі можна побачити "кістяк" системи: наслідування, асоціації, агрегації та композиції. Діаграма класів не показує, як об'єкти взаємодіють у часі, але вона чітко визначає, з яких сутностей система складається та які відносини між ними існують.

3.Які існують канонічні діаграми UML?

UML пропонує широкий спектр діаграм, але серед них можна виділити канонічні, тобто ті, що найчастіше використовуються на практиці. Вони поділяються на дві великі категорії: структурні та поведінкові. До ключових структурних діаграм належать Діаграма класів (статична структура), Діаграма об'єктів (конкретний знімок системи в певний момент) та Діаграма компонентів. Серед поведінкових діаграм найбільш значущими є Діаграма варіантів використання (функціональність системи з точки зору користувача), Діаграма послідовності (взаємодія об'єктів у часі) та Діаграма станів (зміна станів одного об'єкта у відповідь на події). Цей набір дозволяє охопити систему з різних боків.

4.Що таке діаграма варіантів використання?

Діаграма варіантів використання (Use Case Diagram) є основним інструментом для моделювання функціональних вимог до системи на найвищому рівні абстракції. Вона фокусується не на внутрішній будові системи, а на тому, що система повинна робити з точки зору зовнішнього користувача (актора). На цій діаграмі зображуються актори (ролі, які взаємодіють із системою), варіанти використання (функціональні одиниці, що описують певну цінність для актора) та зв'язки між ними. Ця діаграма є ідеальним інструментом для обговорення функціоналу з замовниками та стейкхолдерами, оскільки вона зрозуміла навіть не технічним фахівцям.

5.Що таке варіант використання?

Варіант використання (Use Case) -це не просто окрема функція, а цілісний, корисний для користувача процес, який система виконує для досягнення конкретної цілі певного актора. Він описує послідовність дій у вигляді "успішного сценарію", а також можливі альтернативні шляхи та виняткові ситуації. Наприклад, варіант використання "Замовити книгу" описує весь шлях від пошуку книги до отримання підтвердження замовлення. Кожен варіант використання має чітко визначену мету, яка робить його цінним для актора, що ініціював цю взаємодію.

6.Які відношення існують на діаграмі варіантів використання

Для побудови складних моделей діаграма варіантів використання підтримує кілька типів відношень. Асоціація -це базовий зв'язок між

актором і варіантом використання. Узагальнення (наслідування) показує, що один варіант використання (або актор) є більш спеціалізованою версією іншого. Включення (include) -це обов'язковий зв'язок, що означає, що базовий варіант використання завжди включає в себе поведінку іншого (наприклад, "Оформити замовлення" завжди включає "Авторизувати користувача"). На противагу цьому, розширення (extend) визначає умовний зв'язок, де один варіант використання може бути доданий до іншого в певній точці розширення, якщо виконуються певні умови.

7.Що таке сценарій

Сценарій - це детальний текстовий опис того, як конкретно виконується варіант використання. Він розкриває "атомарну" послідовність кроків, які виконують система та актор для досягнення мети. Зазвичай сценарій описує основний потік подій (ідеальний шлях без помилок) та альтернативні потоки (наприклад, що робити, якщо введено неправильний пароль). Сценарій є основним джерелом інформації для розробників та тестувальників, оскільки саме він формалізує вимоги до поведінки системи на рівні конкретних дій.

8.Що таке діаграма класів

Діаграма класів, як вже згадувалося, є основним інструментом для статичного моделювання структури програмної системи. Вона визначає типи об'єктів, які будуть існувати в системі, інформацію, яку вони будуть зберігати (атрибути), та поведінку, яку вони будуть мати (операції, методи). Крім того, вона детально описує зв'язки між цими класами: хто з ким пов'язаний, в якій кількості (мультиплікативність) та якою мірою. Ця діаграма безпосередньо відображається в програмний код, що робить її центральним артефактом на етапі проектування.

9.Які основні зв'язки між класами

Основними зв'язками між класами в UML є: Асоціація -найзагальніший зв'язок, що показує, що об'єкти одного класу так чи інакше пов'язані з об'єктами іншого. Узагальнення (наслідування) -зв'язок "is-a", де дочірній

клас успадковує структуру та поведінку батьківського. Агрегація та Композиція -це спеціальні форми асоціації, що описують зв'язок "has-a" (частина-ціле). Реалізація-зв'язок між класом та інтерфейсом, що означає, що клас зобов'язується реалізувати всі операції, визначені в інтерфейсі.

10.У чому відмінність композиції від агрегації?

Відмінність між композицією та агрегацією полягає в силі зв'язку "частина-ціле" та в праві власності. Агрегація -це слабкий зв'язок, де частина може існувати незалежно від цілого. Наприклад, "Університет" і "Студент": якщо університет закриється, студенти продовжать існувати. На діаграмі це позначається порожнім ромбом. Композиція -це сильний, нерозривний зв'язок, де життєвий цикл частини повністю залежить від цілого. Наприклад, "Будинок" і "Кімната": при знесенні будинку кімнати також перестають існувати. На діаграмі композиція позначається зафарбованим ромбом на стороні цілого.

11.Що таке нормальні форми баз даних?

Нормальні форми (1NF, 2NF, 3NF, BCNF тощо) - це набір правил і принципів для організації даних у реляційних базах даних. Метою нормалізації є усунення аномалій, таких як надлишковість даних (дублювання інформації), аномалії оновлення, видалення та вставки. Кожна наступна нормальна форма накладає більш строгі обмеження, ніж попередня, забезпечуючи таким чином логічну цілісність, ефективність зберігання та уникнення неслідкових залежностей між полями таблиць.

12.Що таке фізична та логічна модель БД?

Логічна модель бази даних - це абстрактне, концептуальне представлення даних, незалежне від будь-якої конкретної системи управління базами даних (СУБД). Вона фокусується на тому, які дані будуть зберігатися, які сутності, їх атрибути та зв'язки між ними. Фактично, діаграма класів UML може вважатися логічною моделлю. Фізична модель БД- це реалізація логічної моделі в конкретній СУБД. Вона визначає типи даних стовпців, індекси для пришвидшення пошуку, зовнішні ключі, табличні простори та інші технічні деталі, необхідні для фактичного зберігання даних.

13.Який взаємозв'язок між таблицями БД та класами

Між таблицями реляційної бази даних та класами в об'єктно-орієнтованому програмуванні існує тісний зв'язок, який лежить в основі об'єктно-реляційного відображення (ORM). Як правило, один клас відображається на одну таблицю в базі даних. Атрибути (поля) цього класу стають колонками таблиці. Кожен об'єкт (екземпляр класу) відповідає одному рядку в цій таблиці. Зв'язки між класами (асоціації, агрегації) реалізуються за допомогою зовнішніх ключів у таблицях. Наприклад, зв'язок "один-до-багатьох" між класами "Замовлення" та "Товар" буде реалізований через зовнішній ключ "order_id" у таблиці "Товари".